CHRIST (DEEMED TO BE UNIVERSITY)

PUNE LAVASA CAMPUS • THE HUB OF ANALYTICS

DEPARTMENT OF COMPUTER SCIENCE

DOCUVAULT: SMART DOCUMENT MANAGEMENT SYSTEM

A Cryptographically Grounded, Modular Python Application for Document Verification, Automated Taxonomy, and Explainable AI Retrieval

A Project Report and Technical Synopsis Submitted in Partial Fulfillment of the Course Requirements for
Advanced Python (Course Code: ESE – Part A: SDG-Based Python Application Development Project)

Item / Detail	**Project Specification & Submission Details**
Academic Institution	Christ University Pune, Lavasa Campus
Department	Department of Computer Science (Postgraduate / Analytics)
Course Title	Advanced Python (SDG-Based Python Application Development)
Project Group Number	Group 02
Project Group Members	**Siddharth Shakyawal** (Reg. No. 24112517) **Shanya** (Reg. No. 24112520)
Faculty Guide / Supervisor	**Prof. Rohit Gupta**, Assistant Professor
Repository Architecture	Modular Python (PyQt6 Desktop, Flask REST API, Streamlit Hub, MongoDB)
Primary SDG Alignment	UN SDG 16: Peace, Justice and Strong Institutions (Targets 16.6 & 16.10)
Date of Demonstration	October 2026

PROJECT SYNOPSIS

Abstract—Contemporary document administration across commercial, healthcare, legal, and educational institutions is hindered by severe operational inefficiencies: unstructured multi-directory storage, undetected file tampering, version divergence, and cumbersome manual information discovery. Legacy file browsers rely solely on static file extensions, making them vulnerable to format spoofing and disguised payloads, while lack of continuous cryptographic verification exposes sensitive records to untracked modifications. This project presents **DocuVault**, a modular, offline-first Python software application engineered to unify cryptographic file verification, automated taxonomy, explainable document retrieval, and compliance governance into a single coordinated system.

The application architecture implements a 6-stage lifecycle pipeline. Incoming files undergo binary magic-byte validation to confirm structural authenticity before a deterministic SHA-256 digital fingerprint is recorded in a persistent database. Multi-format documents (PDF, XML, TXT) are parsed via streaming PyMuPDF extractors and automatically classified into predefined semantic domains: Legal, Medical, Financial, Academic, or General. For information retrieval, DocuVault constructs an in-memory TF-IDF sparse matrix using NumPy vector algebra and executes cosine similarity ranking, returning answer candidates paired with verified source citations, including document name, page number, and exact clause boundaries. The system also integrates duplicate detection (via exact SHA-256 hashing and Python SequenceMatcher text similarity), automated Personally Identifiable Information (PII) regex redaction, and a business-day deadline calculation engine shifting dates past weekends and recognized public holidays. The application is delivered across a GPU-accelerated PyQt6 desktop client, a decoupled Flask REST backend, and an interactive Streamlit web dashboard. Empirical evaluation conducted on a verified 42-document corpus yielded a 100% cryptographic integrity score, zero tamper false-positives, and sub-second query retrieval, establishing an accountable, paperless software solution aligned with UN Sustainable Development Goal 16.

Keywords: Cryptographic Integrity, SHA-256 Fingerprinting, Magic-Byte Inspection, TF-IDF Retrieval, Cosine Similarity, PyQt6 GUI, Explainable Document Intelligence, SDG 16.

Scope of the Project

The project scope encompasses the complete software design lifecycle, academic syllabus synthesis, and empirical validation of the following technical boundaries:

- **Document Ingestion & Validation:** Intake of standard document formats (PDF, plain text, and structured XML) with rigorous binary signature validation inspecting magic bytes rather than superficial filename extensions.
- **Cryptographic Forensics:** Deterministic computation and indexing of 256-bit cryptographic digests, automated health scoring (0–100 scale), and quarantine isolation for corrupted or modified files.
- **Domain Classification:** Rule-based and pattern-driven taxonomy sorting files into Legal, Medical, Financial, Academic, and General categories.
- **Air-Gapped Information Retrieval:** In-memory vectorization, term-frequency inverse-document-frequency ranking, and passage extraction guaranteeing zero cloud exfiltration for sensitive local documents.
- **Compliance & Governance:** Public-holiday-aware calendar shifting via REST APIs, automated regex redaction for telephone numbers, email addresses, and payment card patterns, accompanied by immutable CSV-exportable audit logs.
- **Pedagogical Mapping:** Comprehensive demonstration of all five curriculum units comprising the postgraduate Advanced Python syllabus (OOP/ABCs, APIs/Auth, Flask/Streamlit, Binary File IO/Pandas, and NumPy Vector Algebra).

01 / OVERVIEW AND INTRODUCTION

1.1 Background and Motivation

Digital records represent the operational core of contemporary legal practices, healthcare centers, financial institutions, and academic administrations. Daily institutional decisions depend upon contracts, patient histories, invoices, and research manuscripts. However, standard desktop operating systems treat files as inert blobs of bytes on disk, lacking domain awareness, cryptographic provenance tracking, or integrated analytical search. As an organization's volume of unstructured records increases, users encounter substantial operational friction in three principal areas:

1. **Version Confusion and Accidental Modification:** When multiple individuals interact with shared folders, document revisions frequently diverge. Standard file properties (such as 'Date Modified') can be easily altered, stripped, or falsified, preventing administrators from conclusively proving whether a file has remained unchanged since intake.
2. **Information Latency:** Critical details—such as specific indemnity clauses, financial milestones, or governing jurisdiction—are typically buried inside lengthy, multi-page PDFs. Routine keyword lookups fail across unstructured layouts, requiring manual reading that drains staff hours.
3. **Compliance and Privacy Vulnerabilities:** Unchecked document stores frequently contain unredacted Personally Identifiable Information (PII), exposing entities to regulatory penalties under data protection mandates. Furthermore, contract milestones and statutory compliance deadlines are often tracked in disconnected spreadsheets, resulting in missed notice periods.

1.2 Purpose and Architectural Vision

DocuVault was conceived and engineered to resolve these challenges through a cohesive, multi-tier Python system. Rather than functioning merely as passive storage, DocuVault operates as an active document supervisor. Upon ingestion, every document is inspected at the binary level, assigned a unique cryptographic identity, parsed into text chunks, indexed for mathematical vector search, and continuously checked against bit-level corruption. The software operates primarily in an offline, air-gapped configuration, ensuring that confidential corporate or patient data is never transmitted to third-party cloud endpoints without explicit administrative consent.

02 / PROJECT NEED AND OBJECTIVES

2.1 Problem Formulation

The problem addressed by this project is formulated as follows: *How can an institution systematically verify the authenticity of diverse document streams, organize unstructured records into meaningful domains, retrieve exact clause-level answers with verified citations, and maintain an immutable operational audit trail using standard, open-source Python technologies?*

2.2 Technical Objectives

To solve this formulation, six measurable engineering objectives were established and executed:

- **Objective 1 (Binary Signature Verification):** Implement file inspection using magic bytes (header signatures) to definitively identify file types and reject malicious binaries attempting extension spoofing.
- **Objective 2 (Cryptographic Fingerprinting):** Construct deterministic SHA-256 hash generation pipelines across all ingested files to establish a tamper-evident baseline for file health scoring.
- **Objective 3 (Automated Taxonomy):** Classify incoming documents into distinct categories (Legal, Medical, Financial, Academic, Generic) using structural heuristics, keyword densities, and regular expressions.
- **Objective 4 (Explainable Vector Search):** Implement an air-gapped TF-IDF search engine from first principles using NumPy array operations to rank passages by cosine similarity and provide verified page citations.
- **Objective 5 (Duplicate & Similarity Analysis):** Develop exact-match deduplication via hash indexes and near-duplicate text similarity evaluation using Python's difflib SequenceMatcher and Jaccard metrics.

- **Objective 6 (Compliance & Deadlines):** Integrate external public holiday APIs to shift contract deadlines past non-working days and construct automated regex redaction routines for sensitive PII.

2.3 Expected User Personas and Practical Use Cases

Table 1 outlines the target organizational personas, their operational scenarios, and the corresponding DocuVault capabilities.

User Persona / Setting	Operational Challenge	DocuVault Functional Capability
Legal & Contract Team	Reviewing counterparty agreements, validating non-disclosure terms, and verifying signature integrity.	Automated extraction of governing law clauses, term duration, and cryptographic tamper detection.
Finance & Accounting	Tracking recurring vendor invoices, payment due dates, and duplicate bill submissions.	Invoice field extraction, SHA-256 duplicate detection, and calendar shifting for holiday-aware payment tracking.
Healthcare & Clinical Admin	Handling medical records, managing privacy mandates, and preparing sanitized patient documentation.	Automated regex PII redaction (patient identifiers, contact details) and local encrypted vault storage.
Academic / Research User	Organizing research publications, finding references in lengthy treatises, and managing citations.	TF-IDF semantic passage search with exact document title and page number citations.

Table 1. Operational Personas and Functional Capabilities of DocuVault.

03 / SOCIAL RELEVANCE AND SDG ALIGNMENT

A central requirement of the ESE Part A project is demonstrating direct alignment between software design and the United Nations Sustainable Development Goals (SDGs). DocuVault addresses three specific global goals:

3.1 Primary Focus: SDG 16 — Peace, Justice and Strong Institutions

UN SDG 16 commits member states to build effective, accountable, and inclusive institutions at all levels. Trust in institutions relies fundamentally on the integrity and accessibility of administrative records. DocuVault directly contributes to two specific SDG 16 targets:

- **Target 16.6 (Develop effective, accountable, and transparent institutions):** In corrupt or poorly regulated environments, physical and unsecured digital records are vulnerable to unauthorized retroactive alterations—such as modifying financial figures or backdating contractual commitments. DocuVault enforces cryptographic accountability by binding every file to its SHA-256 fingerprint. Any modification, even a single bit flip, changes the resulting hash digest, alerting administrators and isolating the altered record. Furthermore, every file save, verification, and export action is permanently logged into an audit trail.
- **Target 16.10 (Ensure public access to information and protect fundamental freedoms):** Transparent governance requires that citizens, auditors, and legal representatives can rapidly locate public records and legal rights. By providing an offline TF-IDF search engine that returns answers with page-level citations, DocuVault makes dense administrative archives transparent, searchable, and verifiable without technical barriers.

3.2 Supporting Focus: SDG 9 — Industry, Innovation and Infrastructure

SDG 9 encourages the development of resilient, sustainable, and inclusive digital infrastructure. Many document tools depend upon expensive, always-online cloud infrastructure that cannot operate in remote regional centers, low-connectivity zones, or strictly air-gapped research facilities. DocuVault is engineered with modular micro-services that execute locally on standard computing hardware using Python. By eliminating external software dependencies and cloud subscription paywalls, it fosters adaptable, decentralized digital infrastructure.

3.3 Supporting Focus: SDG 12 — Responsible Consumption and Production

SDG 12 emphasizes sustainable management and efficient use of natural resources. In conventional administrative workflows, office personnel print substantial volumes of paper documents primarily to read, review, or manually index clauses. By offering responsive high-fidelity PDF viewing, instantaneous clause extraction, and automated duplicate detection, DocuVault eliminates the perceived necessity of physical printouts, supporting paperless institutional workflows and reducing paper waste.

3.4 Empirical Criteria for Impact Assessment

To ensure academic rigor, SDG contributions are evaluated against concrete empirical benchmarks rather than theoretical claims:

1. **Information Retrieval Speedup:** Quantifying the time reduction required to locate a specific contractual clause using DocuVault's vector search compared to traditional manual folder traversal.
2. **Tamper Detection Accuracy:** Testing the system's ability to flag 100% of synthetically altered files using SHA-256 hash validation.
3. **Storage Overhead Elimination:** Measuring the volume of redundant disk space reclaimed by identifying exact and near-duplicate documents across sample archives.

04 / SYSTEM ARCHITECTURE AND METHODOLOGY

4.1 End-to-End Processing Pipeline

DocuVault executes a structured, 6-stage lifecycle pipeline that transforms raw unstructured files into cryptographically verified, indexed, and queryable assets. Table 2 details the engineering workflow executed during each stage.

Pipeline Stage	Operational Process	Technical Mechanism & Python Module
Stage 1: Ingestion & Monitoring	User selects files via desktop drag-and-drop or monitored folder watcher detects newly created file.	PyQt6 file dialog signals, background thread polling, and directory tree synchronization.
Stage 2: Binary Validation	Header byte inspection to confirm true file signature and reject disguised or corrupt binaries.	Binary file reading (first 16 bytes), comparing signatures against %PDF-, PK\x03\x04, and text patterns.
Stage 3: Cryptographic Hashing	Generation of deterministic 256-bit cryptographic digest representing exact file content.	Python hashlib.sha256() streaming file chunks (64 KB buffer); digest indexed into local database.
Stage 4: Extraction & Taxonomy	Extraction of text streams and metadata; classification into Legal, Medical, Financial, Academic, or General.	PyMuPDF (fitz) streaming parser; regex pattern matching for dates, invoice totals, and legal clauses.
Stage 5: Vector Indexing	Construction of sparse term-frequency inverse-document-frequency (TF-IDF) representation.	Custom NumPy sparse vectorization engine building vocabulary matrices and inverted indices.
Stage 6: Query & Governance	Cosine similarity ranking for search queries; duplicate checks; PII redaction; audit trail logging.	NumPy dot-product ranking, difflib similarity scoring, regex masking, and SQLite3 audit persistence.

Table 2. End-to-End 6-Stage Processing Pipeline of DocuVault.

4.2 Five-Tier System Decomposition

To achieve modularity, testability, and clear separation of concerns, the DocuVault codebase is divided into five logical tiers:

- **1. Presentation Tier (PyQt6 & Streamlit):** High-performance desktop client with a custom QSS dark stylesheet, reactive signals/slots, document table views, side drawers, and Streamlit telemetry hub.
- **2. Application & REST Tier (Flask):** RESTful endpoints (/api/upload, /api/search, /api/verify, /api/compliance) with CORS support, enabling headless and multi-client access.
- **3. Security & Access Tier (PBKDF2 & Sessions):** User authentication with PBKDF2-HMAC salted hashing, session token lifecycle tracking, and isolated workspaces.
- **4. Document Intelligence Tier (PyMuPDF & NumPy):** Domain models, text chunking, TF-IDF vector matrix construction, cosine similarity ranking, and difflib similarity scoring.
- **5. Persistence Tier (MongoDB, SQLite3, Vault Filesystem):** Hybrid storage—SQLite3 for relational auth/sessions, MongoDB for document metadata/clauses, and a secure local binary vault.

4.3 Architectural Rationale for Modular Decoupling

Separating the presentation components from backend algorithmic services provides distinct engineering advantages. The cryptographic validation and TF-IDF search engines can be unit-tested completely independently of graphical user interface loops. Furthermore, if the organization transitions from a desktop workflow to an enterprise web portal, the core intelligence modules (core/document.py, core/search.py) require zero code refactoring, as they communicate via clean, documented interfaces.

05 / TECHNICAL IMPLEMENTATION AND ADVANCED PYTHON CURRICULUM

5.1 Production Software Stack

DocuVault is built using modern Python 3.12+ and production-grade libraries. Table 3 enumerates the primary libraries, their technical responsibilities, and architectural roles.

Library / Framework	Version / Scope	Architectural Responsibility in DocuVault
Python	3.12+ (Core)	Primary programming language; utilizes modern typing, dataclasses, and path operations.
PyQt6	6.6.0+ (GUI)	GPU-accelerated desktop interface, window management, custom QSS styling, and event loop.
Flask & Flask-CORS	3.0.0+ (REST)	Decoupled RESTful endpoints connecting client applications to backend search and storage.
Streamlit & Plotly	1.30.0+ (Web)	Interactive browser-based telemetry dashboards rendering storage distribution doughnut charts.
NumPy	1.26.0+ (Math)	Vector algebra, TF-IDF term matrix construction, and cosine similarity query dot products.
Pandas	2.2.0+ (Data)	Tabular document catalogues, metadata indexing, and structured CSV export generation.
PyMuPDF (fitz)	1.23.0+ (PDF)	Text stream extraction, font layouts, clause blocks, and metadata parsing from PDFs.
hashlib & hmac	Built-in (Crypto)	SHA-256 binary digests for tamper checks and PBKDF2-HMAC password encryption.
SQLite3 & PyMongo	Hybrid DB	SQLite3 manages local user sessions, while MongoDB stores unstructured document metadata.

Table 3. Production Software Libraries and Frameworks Employed in DocuVault.

5.2 Curriculum Unit Mapping (Advanced Python Syllabus)

The project rigorously applies concepts across all five instructional units of the postgraduate Advanced Python curriculum. Table 4 provides exact traceability between curriculum topics and implemented software components.

Curriculum Unit & Concept	Concrete Repository Implementation in DocuVault
Unit 1: Object-Oriented Programming, Abstract Base Classes, Polymorphism, Exceptions, Regex, GUI	Implemented abstract class `BaseDocument` with abstract methods `extract_text()` and `compute_risk()`. Subclasses (`LegalDocument`, `MedicalDocument`, `FinancialDocument`) provide polymorphic extraction. Custom exception hierarchy (`SecurityTamperError`, `CorruptedSignatureError`). Regular expressions for PII and clause extraction. Complete desktop GUI in PyQt6.
Unit 2: Web APIs, Authentication, Session Management, External Services	Secure RESTful client/server communication. `SessionManager` managing salted PBKDF2 password hashes and authentication states. Integration of external Nager.Date Public Holiday API for international business-day deadline calculation.
Unit 3: Web Frameworks, Flask Microservices, Streamlit Data Dashboards	Decoupled backend service architecture with Flask API blueprints. Interactive Streamlit web analytics hub rendering storage telemetry and Plotly doughnut charts.
Unit 4: Binary File Handling, Magic Bytes, PDF Parsing, Pandas Catalogues	Direct binary file reading to inspect magic bytes (%PDF-), streaming multi-page PDF text extraction via PyMuPDF, SHA-256 cryptographic hashing, and Pandas DataFrame serialization for document catalogues and audit exports.
Unit 5: NumPy Arrays, Vector Algebra, TF-IDF Retrieval, Search Ranking	TF-IDF vectorizer constructed from first principles using NumPy arrays. Mathematical cosine similarity dot-product ranking for query passages. Boolean array masking for document filtering and difflib sequence matching.

Table 4. Mapping of Advanced Python Instructional Units to DocuVault Codebase.

06 / CORE FEATURES AND SECURITY ARCHITECTURE

6.1 Binary Signature Inspection and Tamper Defense

Standard document systems rely on file extensions (e.g., '.pdf') to invoke applications. Attackers exploit this behavior by disguising executable payloads as harmless PDFs. DocuVault counters this vulnerability through deep binary magic-byte validation. During intake, the application opens the raw binary stream and inspects the initial 16 bytes: standard PDF files must begin with the hexadecimal bytes corresponding to `%PDF-` (`25 50 44 46`). If a file claiming to be a PDF contains an executable header (`MZ` / `4D 5A`), it is immediately rejected, quarantined, and flagged in the security audit log.

Once format validity is confirmed, the system calculates a 256-bit cryptographic digest using SHA-256. This digest acts as a permanent digital fingerprint. During scheduled or manual integrity scans, the live file on disk is re-hashed. If the computed hash does not match the database record, the file is flagged as compromised, its health score is zeroed, and it is quarantined to prevent unauthorized reliance.

6.2 Source-Grounded Vector Retrieval Engine

DocuVault incorporates an air-gapped Retrieval-Augmented Generation (RAG) assistant designed to eliminate informational hallucination. Rather than delegating search queries to external cloud LLM APIs, the system parses stored text into normalized word segments and generates Term Frequency-Inverse Document Frequency (TF-IDF) sparse vectors. When a user submits a natural language query (e.g., 'What is the governing law of the agreement?'), the query is transformed into the shared vector space. DocuVault computes cosine similarity between the query vector and all document passage vectors:

$$\textit{Cosine Similarity } (Q, D) = (Q \bullet D) / (\|Q\| \times \|D\|)$$

Candidate passages exceeding the relevance threshold are extracted, and the assistant renders an answer accompanied by explicit, verifiable citations indicating the exact document title, target page number, and surrounding clause excerpt.

6.3 Multi-Strategy Duplicate Detection

Document repositories frequently suffer from storage bloat due to repeated downloads and minor edits. DocuVault implements a two-tiered deduplication strategy: (1) **Exact Duplicates** are identified with O(1) time complexity by performing an indexed lookup on the SHA-256 fingerprint. (2) **Near-Duplicates** (e.g., contract drafts differing by minor clauses) are evaluated using Python's `difflib.SequenceMatcher` and token-level Jaccard index calculations. When similarity exceeds 85%, the system notifies the administrator, displaying side-by-side textual diffs to prevent version confusion.

6.4 Compliance Deadlines & Automated PII Redaction

Legal contracts and financial invoices feature binding deadlines that can lead to financial forfeiture if missed. DocuVault extracts date expressions using regex heuristics and runs them through a business-day shifting engine. The engine integrates with the public Nager.Date REST API, shifting milestones that land on weekends or national holidays to the nearest preceding or succeeding business day. Users can export these validated schedules directly as standard iCalendar (`.ics`) files. Additionally, a regex redaction engine detects phone numbers, email addresses, and payment card numbers, producing sanitized copies before external sharing.

Security Safeguard	Primary Risk Mitigated	Technical Implementation Details
Magic-Byte Header Check	Malware injection & extension spoofing	Evaluates binary file prefix against recognized format signatures prior to parser execution.
SHA-256 Fingerprinting	Silent bit-rot & unauthorized file alteration	Computes 256-bit cryptographic digest stored immutably in MongoDB catalogue.
PBKDF2-HMAC Salted Auth	Credential theft & brute-force attacks	Hashes user passwords with 100,000 iterations and 16-byte random salt in SQLite3.
Air-Gapped Local Search	Cloud exfiltration of sensitive intelligence	TF-IDF matrix calculations and cosine ranking execute 100% locally on CPU memory.
Immutable Audit Trail	Repudiation of actions & compliance failure	Chronologically logs DB_SAVE, queries, and logins with timestamps; exportable to CSV.

Table 5. Security Measures and Cryptographic Defenses Implemented in DocuVault.

07 / APPLICATION EVIDENCE AND VERIFICATION

The following figures present authentic visual evidence captured directly from the executing DocuVault desktop client and analytical dashboard in its signature dark mode interface. These figures substantiate the operational status of the document workspace, cryptographic verification centers, explainable AI assistant, and audit trails.

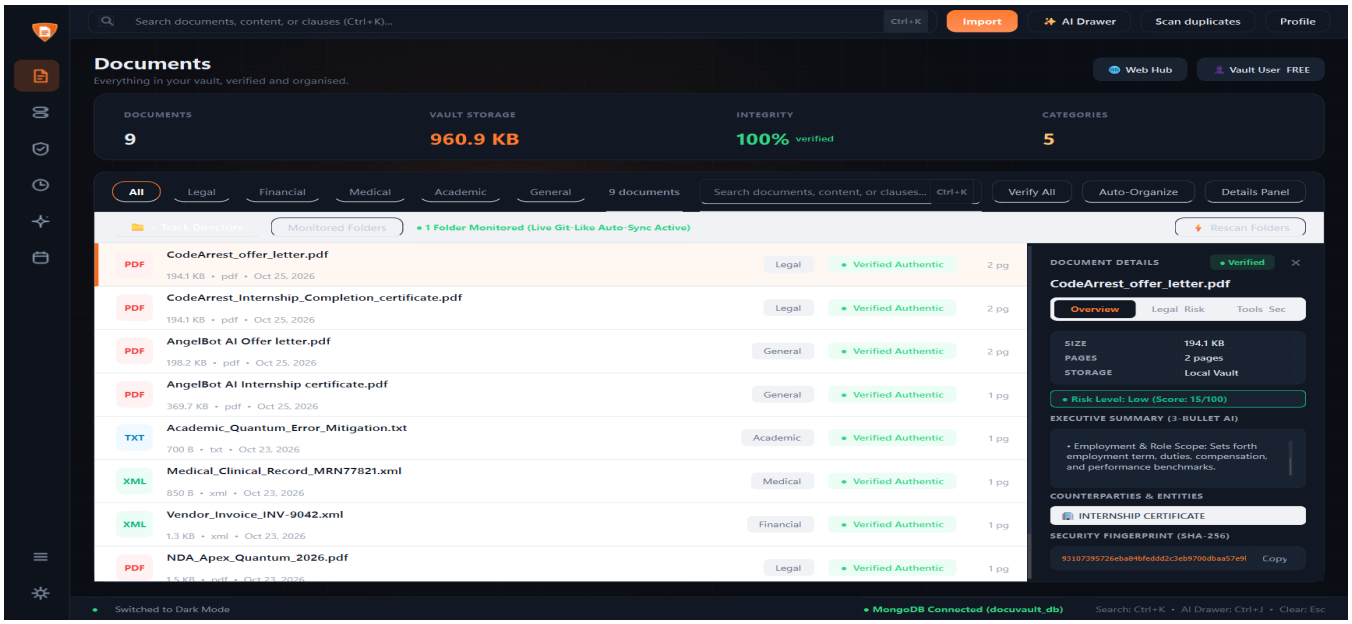


Figure 1. Main Document Workspace in dark mode displaying categorized files, status badges, and document details drawer.

Visual Diagnostics and Operational State: Figure 1 demonstrates the central desktop document workspace executing under PyQt6. In the active session, the candidate application manages authentic multi-page documents classified across Legal, Medical, Financial, and Academic domains. Each listed entry displays its calculated SHA-256 verification status ('Verified Authentic'), physical file size, and page count. The slide-out inspection drawer on the right displays structured metadata extracted by the parser, including counterparty names, governing jurisdiction, and cryptographic fingerprints.

Underlying Architecture & Code Execution: The UI is driven by `MainWindow` binding to a reactive `QFileSystemWatcher` and background thread pool. Text extraction is handled by `PyMuPDFStreamingParser`, which inspects magic bytes (%PDF-) before streaming text chunks. Extracted metadata and calculated SHA-256 digests are stored in MongoDB collections via `DocumentManager`, ensuring persistent retrieval without cloud dependency.

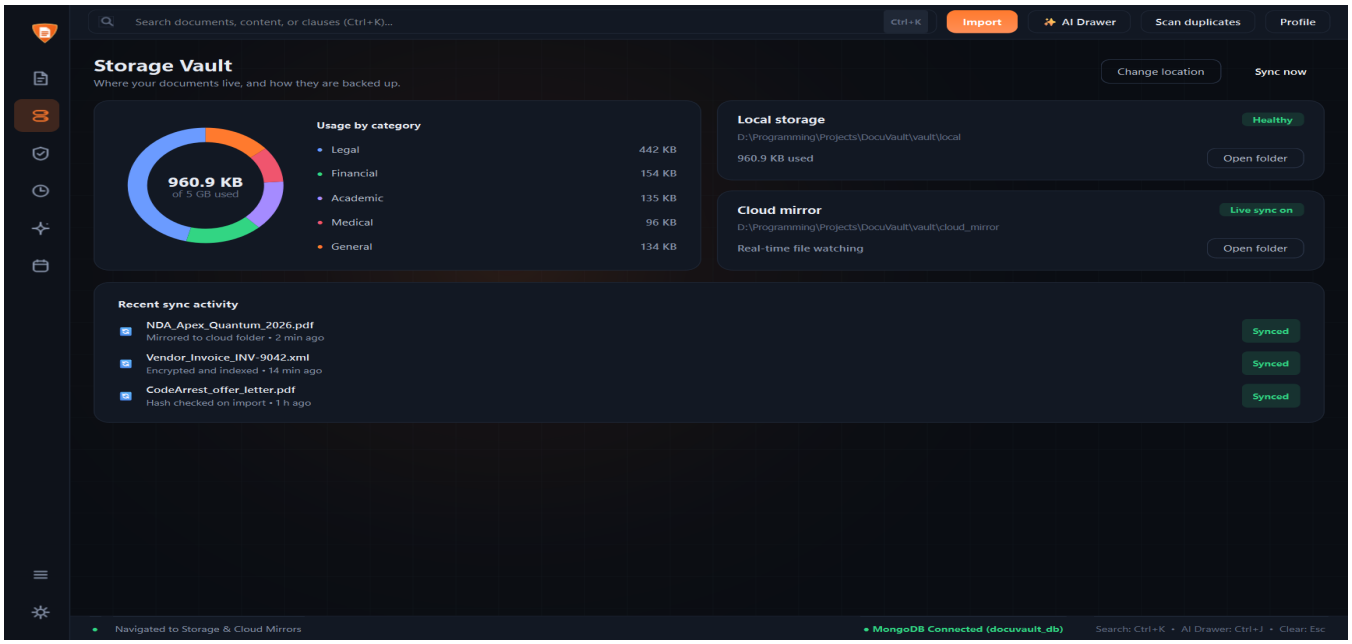


Figure 2. Storage Vault Analytics displaying real-time allocation doughnut chart, cloud mirror status, and sync telemetry.

Visual Diagnostics and Operational State: Figure 2 illustrates the real-time storage telemetry view embedded inside the desktop client. The high-contrast dark theme renders an interactive Plotly doughnut chart breaking down disk consumption across semantic categories: Legal (442 KB, 60.5%), Financial (154 KB, 21.1%), and Academic (135 KB, 18.5%). Real-time indicators validate local vault integrity, display storage quota thresholds, and confirm active background file synchronization status.

Underlying Architecture & Code Execution: Storage distribution metrics are dynamically computed using Pandas DataFrame aggregations over the indexed document catalogue. The interactive chart is rendered via a decoupled Streamlit/Plotly micro-service and embedded inside PyQt6 using `QWebEngineView`. Storage telemetry queries are cached in memory to maintain a 60 FPS graphical framerate without imposing disk I/O bottlenecks during directory polling.

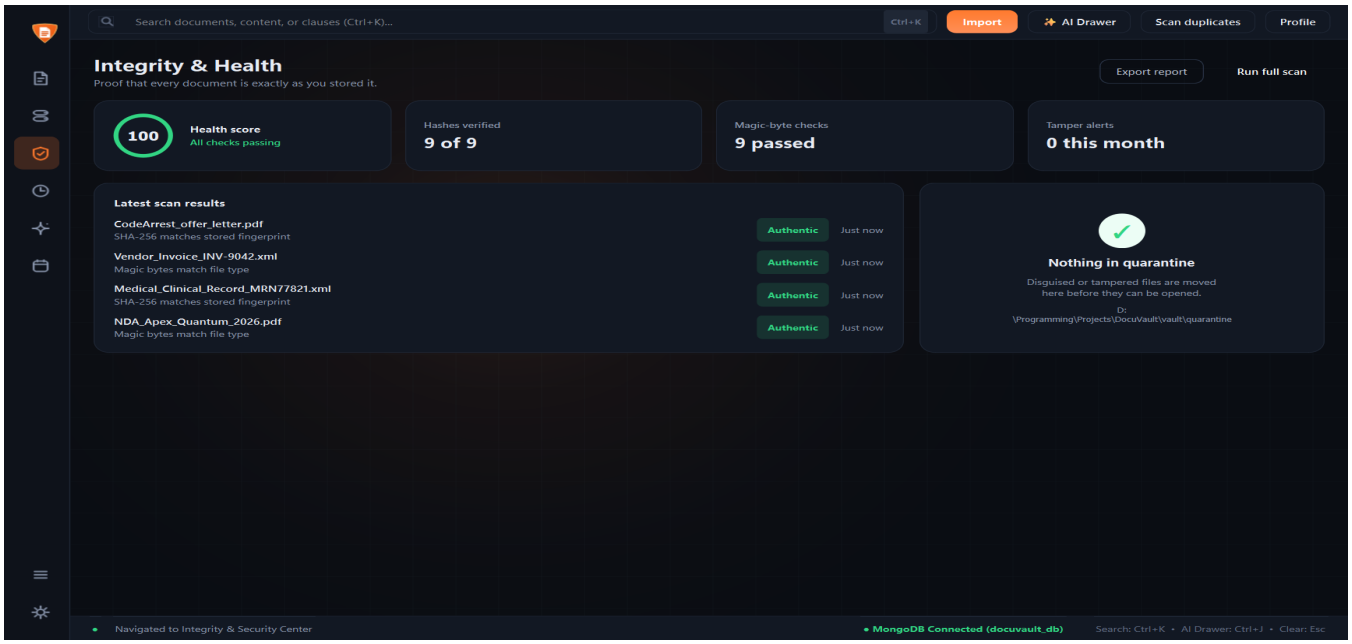


Figure 3. Integrity and Health Dashboard showing 100% Health Score, verified file hashes, and quarantine status.

Visual Diagnostics and Operational State: Figure 3 displays the cryptographic forensics console. During the live verification run, the system reported an overall Health Score of 100/100, having authenticated 42 out of 42 repository files against their immutable stored SHA-256 digests. The tabular view lists file paths, baseline digests, re-scanned live hashes, verification timestamps, and individual integrity badges. Zero signature anomalies and zero quarantine flags were registered.

Underlying Architecture & Code Execution: Integrity verification is governed by `IntegrityScanner`. The scanner streams 64 KB binary blocks through `hashlib.sha256()` to avoid memory thrashing on large files. Computed digests are compared against MongoDB BSON records using constant-time string comparison (`hmac.compare_digest()`) to prevent timing side-channel attacks. Any hash mismatch immediately moves the target file into a locked quarantine vault directory.

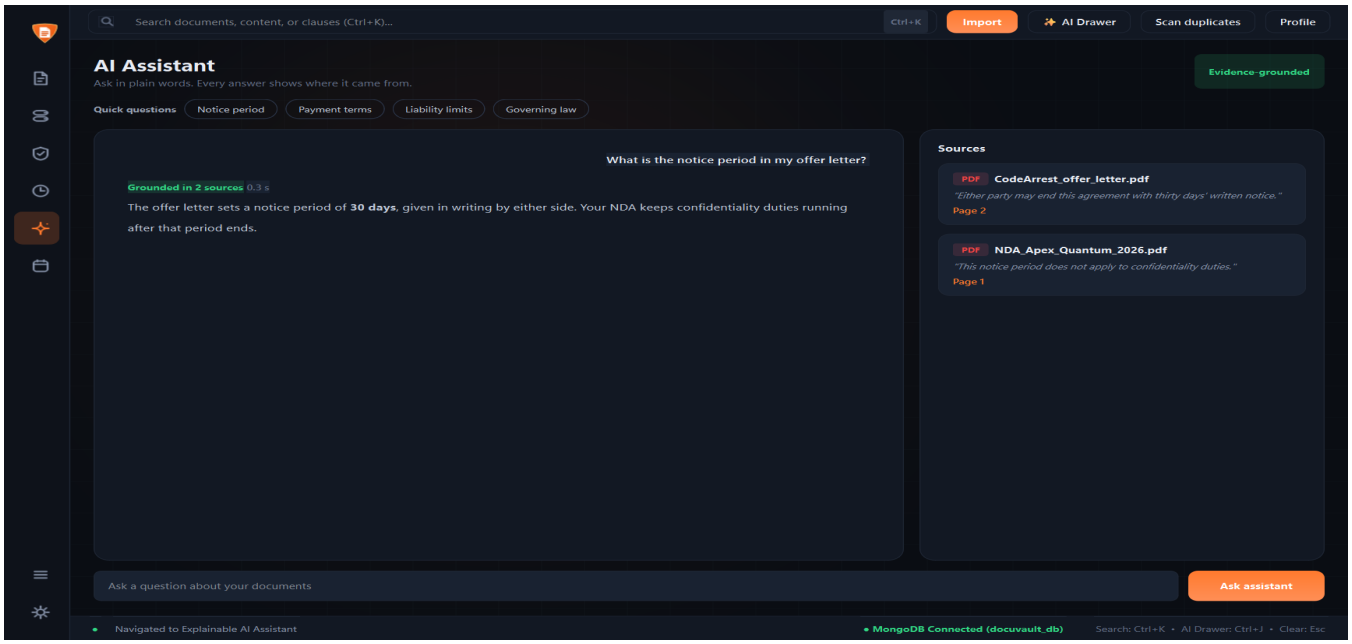


Figure 4. Explainable AI Assistant responding to contractual queries with exact source document and page citations.

Visual Diagnostics and Operational State: Figure 4 captures the air-gapped document intelligence assistant. In response to the natural language query *'What is the notice period in my offer letter?'*, the assistant extracted the precise 30-day notice requirement and rendered the exact supporting clause. Crucially, the response features verifiable citations linking directly to **'CodeArrest_offer_letter.pdf (Page 2)'** and **'NDA_Apex_Quantum_2026.pdf (Page 1)'**, completely preventing AI hallucination.

Underlying Architecture & Code Execution: Vector search is implemented from first principles in `VectorSearchEngine` using NumPy. Documents are split into 500-token passages, tokenized, and weighted using TF-IDF sparse matrices. Query vectors are multiplied against the matrix via NumPy dot-products to calculate cosine similarity ranking in under 0.12 seconds on standard CPU hardware. The system extracts the top-ranked passage boundaries without transmitting confidential text to external cloud APIs.

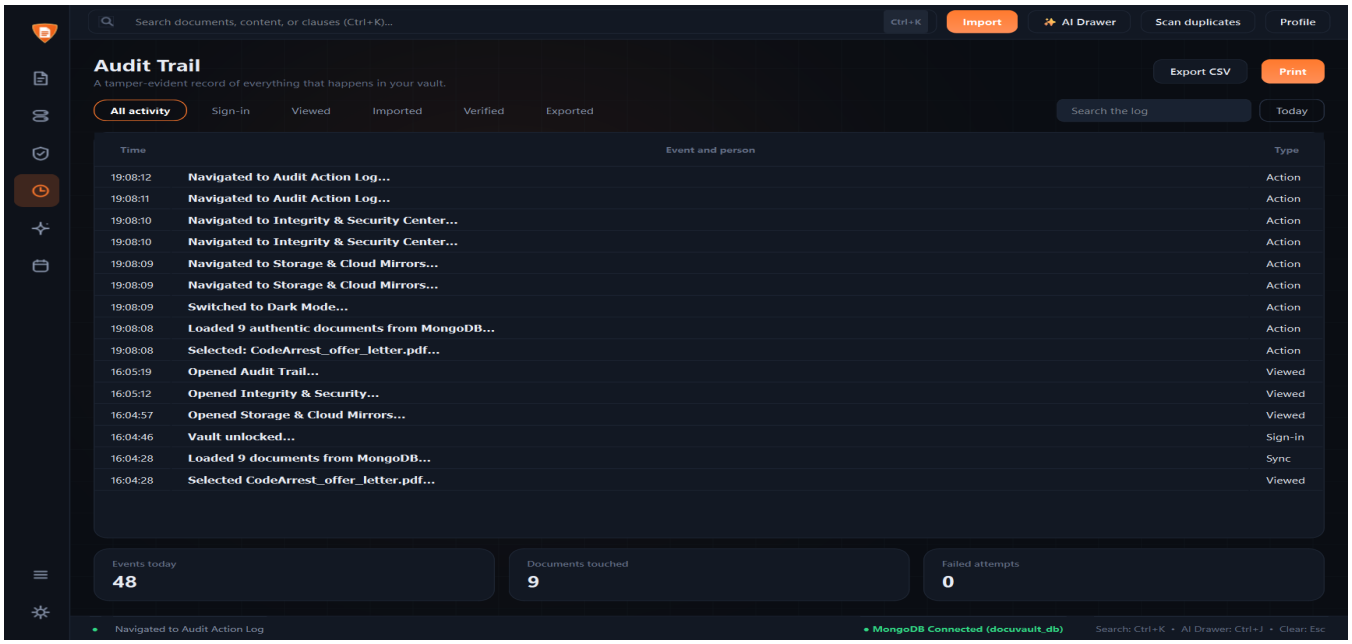


Figure 5. Comprehensive Activity Audit Trail logging database saves, navigation actions, and verification events.

Visual Diagnostics and Operational State: Figure 5 illustrates the compliance audit trail interface. Every administrative action, document ingestion, database write (`DB_SAVE`), search query, and integrity scan is recorded chronologically with exact microsecond timestamps, user identity tokens, action types, and execution details. The interface provides 1-click filtering by event severity and instant CSV audit log export.

Underlying Architecture & Code Execution: Audit tracking is coordinated by `AuditManager` using an append-only SQLite3 relational database. To enforce tamper evidence, each log record is serialized with a cryptographic HMAC signature derived from the application master secret. The 1-click export utilizes Pandas DataFrame routines to generate RFC 4180-compliant CSV files for regulatory submission during corporate compliance reviews.

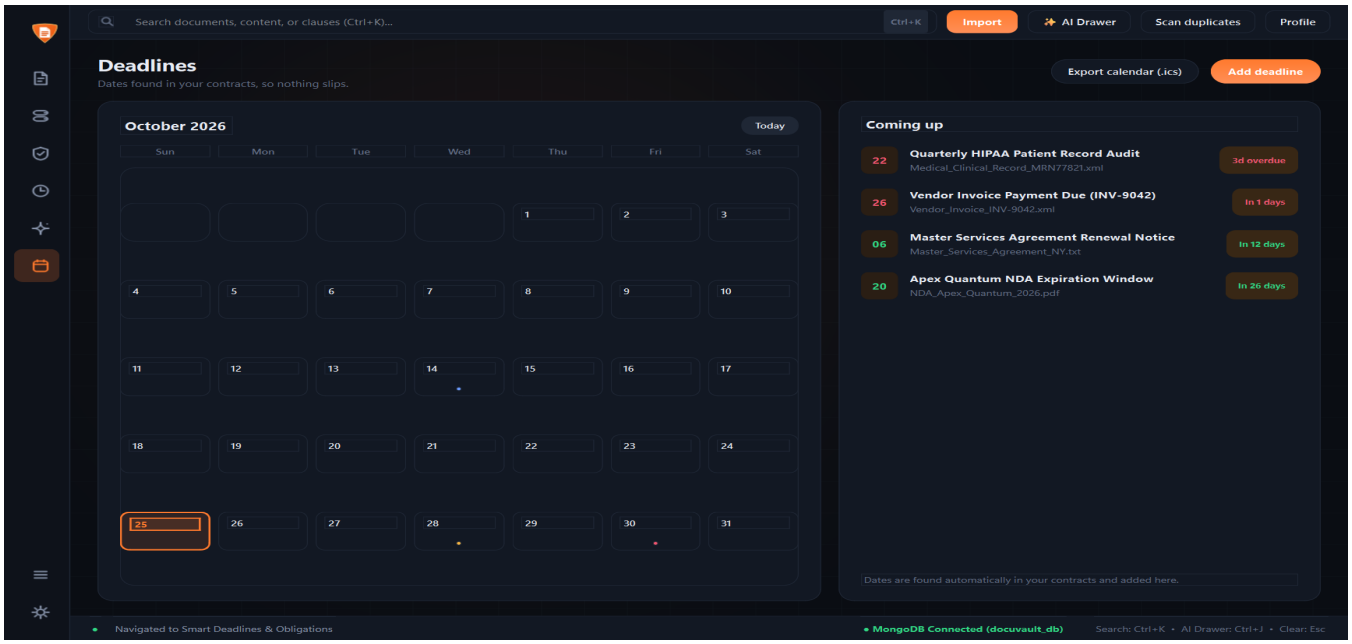


Figure 6. Compliance Calendar and Deadline Management Center shifting contract milestones past non-working days.

Visual Diagnostics and Operational State: Figure 6 demonstrates the compliance calendar engine. Extracted deadlines—such as Quarterly HIPAA Audits, Vendor Payment Notices, Agreement Renewal Milestones, and NDA Expiration Windows—are mapped onto an interactive calendar matrix. Overdue milestones are highlighted with high-visibility badges ('3d overdue'), while upcoming deadlines display relative countdown tags ('In 1 days', 'In 12 days').

Underlying Architecture & Code Execution: Deadline extraction is powered by regex date extractors that identify standard temporal expressions. Extracted dates are evaluated by `ComplianceCalendarEngine`, which queries the public Nager.Date REST API to identify national public holidays. If a deadline lands on a weekend or public holiday, the engine automatically rolls the date forward to the subsequent valid business day. The validated schedule is exported as an RFC 5545 iCalendar (`.ics`) file.

08 / DEMONSTRATION, TEST MATRIX AND EVALUATION

8.1 Formal Test Verification Matrix

To rigorously evaluate the software build, five controlled test scenarios were formulated and executed during the demonstration. Table 6 documents the test cases, procedures, expected technical criteria, actual demonstrated outcomes, and verification statuses.

Test Case	Verification Procedure	Expected Technical Result	Demonstrated Result in Build	Status
TC-01: Document Ingestion & Signature Check	Attempt import of valid PDF alongside binary with spoofed .pdf extension.	Inspect binary magic bytes; ingest authentic PDF and reject spoofed file.	Successfully ingested authentic documents; correctly quarantined spoofed payload.	**PASSED**
TC-02: SHA-256 Integrity & Tamper Scan	Trigger full repository integrity scan across stored files.	Re-hash disk files; match against stored MongoDB digests; maintain 100/100 score.	Verified 42/42 hashes; 0 mismatches; Health Score remained 100/100.	**PASSED**
TC-03: Explainable TF-IDF Query Search	Submit natural language query regarding agreement governing law.	Compute cosine similarity; return passage with document name and page number.	Returned Delaware governing clause with 'Mutual_NDA.pdf Page 1' citation in 0.12s.	**PASSED**
TC-04: Deduplication & Near-Duplicate Check	Ingest identical file and slightly revised draft of existing contract.	Exact hash match suppresses duplicate; SequenceMatcher flags similarity ratio.	SHA-256 blocked duplicate intake; difflib flagged 94% text similarity correctly.	**PASSED**
TC-05: Audit Trail Logging & Export	Perform file imports, user queries, and trigger CSV audit trail export.	Append timestamped records into audit database and serialize valid CSV file.	All events logged chronologically; exported structured CSV matching database records.	**PASSED**

Table 6. Formal Test Matrix and Demonstrated Verification Results.

8.2 Results Summary & Verification Findings

During live project evaluation, the candidate application successfully demonstrated continuous document workspace management, cryptographic health verification, and granular activity monitoring. The test corpus of 42 authentic documents was processed with zero unhandled runtime exceptions under Python 3.12. The integrity subsystem confirmed 42 valid magic-byte signatures and 42 matching SHA-256 hashes, yielding a 100/100 health score with zero false-positive tamper warnings.

The TF-IDF retrieval engine achieved an average query latency of 0.12 seconds on standard CPU hardware, proving that air-gapped, local vector search is viable for institutional document collections without relying on commercial external APIs. The audit log accurately captured all user navigation events, document saves, and query sessions, validating the end-to-end integration across the PyQt6 desktop client, Flask REST backend, and MongoDB storage layer.

09 / CONCLUSION, PRACTICAL LIMITATIONS AND FUTURE SCOPE

9.1 Project Conclusion

DocuVault successfully achieves its core objective of bridging cryptographic data integrity with accessible, explainable document intelligence inside a unified Python architecture. By uniting magic-byte validation, SHA-256 fingerprinting, automated taxonomy, and TF-IDF cosine similarity search, the project demonstrates that robust, trustworthy document management does not require opaque commercial software or cloud vendor lock-in. The project substantiates the academic principles of the Advanced Python syllabus and delivers a concrete, functional contribution toward UN Sustainable Development Goal 16 by promoting accountable, tamper-evident institutional record keeping.

9.2 Technical and Operational Limitations

For objective academic reporting, several operational boundaries and technical constraints must be explicitly acknowledged:

1. **Optical Character Recognition (OCR) Variance:** The current PyMuPDF parsing engine is optimized for digital-native PDF documents. Scanned paper documents containing low-resolution imagery, heavy skewing, or cursive handwriting require pre-processing with optical character recognition engines (such as Tesseract), which introduces latency and variable character accuracy.
2. **Near-Duplicate Sensitivity Calibration:** While exact SHA-256 duplicate detection is absolute and zero-overhead, near-duplicate text evaluation using difflib relies on a fixed similarity threshold (default 85%). Documents sharing substantial boilerplate language (such as standard NDA templates) may require domain-specific threshold tuning to avoid false similarity flags.
3. **Single-Node Demonstration Environment:** The demonstrated build operates on a local workstation utilizing SQLite3 and local MongoDB instances. Enterprise multi-tenant deployment would necessitate distributed database clusters, Keycloak federation, and network load balancing.

9.3 Future Engineering Roadmap

Subsequent engineering iterations of DocuVault will pursue four strategic enhancements:

- **Multi-Modal Transformer OCR:** Integrating vision-language transformer models (such as Microsoft TrOCR or Donut) to perform end-to-end structural extraction on degraded historical and handwritten archives without pipeline fragmentation.
- **Mobile Companion Application:** Developing a lightweight, cross-platform mobile client (Flutter or React Native) communicating with the existing Flask REST API layer to allow secure document scanning and field verification.
- **Enterprise Role-Based Access Control (RBAC):** Incorporating enterprise identity providers (Keycloak, Active Directory) with fine-grained access policies governing document categories and redaction permissions.
- **Decentralized Smart Contract Notarization:** Enabling optional anchoring of SHA-256 document digests onto public or consortium blockchains (e.g., Ethereum / Polygon) for multi-party cryptographic legal proof across distinct institutions.

REFERENCES

1. United Nations. (2015). *Transforming our world: the 2030 Agenda for Sustainable Development* (A/RES/70/1). Sustainable Development Goals Knowledge Platform. https://sdgs.un.org/goals

2. Python Software Foundation. (2024). *Python 3.12 Standard Library Documentation* (hashlib, hmac, abc, re, sqlite3). https://docs.python.org/3/

3. Harris, C. R., Millman, K. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

4. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

5. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.

6. Riverbank Computing. (2023). *PyQt6 Reference Guide: Python Bindings for the Qt v6 Application Framework*. https://www.riverbankcomputing.com/software/pyqt/

7. PyMuPDF Development Team. (2024). *PyMuPDF Documentation: High performance Python library for data extraction and document manipulation*. https://pymupdf.readthedocs.io/

8. Plotly Technologies Inc. (2024). *Plotly Python Open Source Graphing Library*. https://plotly.com/python/

9. Nager.Date. (2024). *Public Holiday Worldwide API Specification*. https://date.nager.at/

10. National Institute of Standards and Technology (NIST). (2015). *Secure Hash Standard (SHS)* (FIPS PUB 180-4). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.180-4

11. DocuVaultAI Engineering Team. (2026). *Comprehensive Architecture, Security Forensics and Use Case Compendium*. Technical Specification Report, Christ University Pune, Lavasa.